

MyResume- Phase 4: Optimisation & Automation

Overview

My Resume is an online, cloud-deployed version of my portfolio. This project focuses on the practical application of Microsoft Azure services and demonstrating foundational cloud architecture and deployment skills. This project is developed as the kickstart for my Cloud Computing journey.

Phase 4 of this project involves deploying the serverless version of the project with Terraform. The backend Azure Function and Azure Cosmos DB is removed while the data is bundled together with the React build.

This document will serve as the overview for MyResume Phase 4: Optimisation & Automation.

The Tech Stack:

- Frontend: React with Typescript (Deployed over Azure Static Web App)

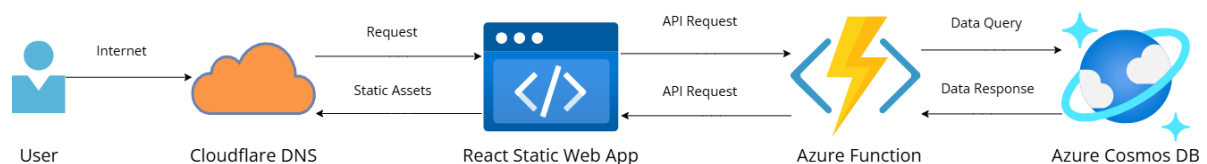
Contents

Overview	1
Architecture	3
Terraform	5
Azure Static Web App.....	7
Playwright	8
Deployment Workflow	10
Conclusion	12

Architecture

BEFORE – The serverless architecture is a simple flow:

- DNS Resolution & Edge Caching: When a user navigates to the domain, Cloudflare resolves the request and directs traffic to the Azure Static Web App (SWA) origin.
- API Trigger: The React frontend initiates an API request to the Azure Function URL, which executes the serverless backend logic.
- Data Retrieval: The Azure Function establishes a connection to Azure Cosmos DB to perform the requested data query.
- JSON Response: Cosmos DB returns the query results to the Function, which then sends the data back to the React frontend as a structured JSON response.
- Client-Side Rendering: Finally, React processes this JSON data to dynamically update the UI for the user.

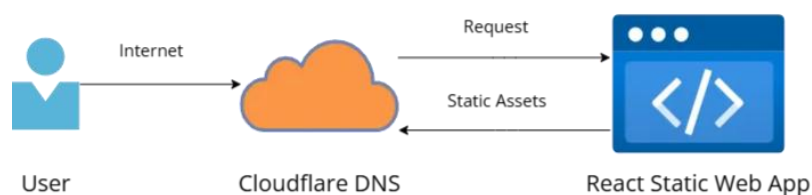


After - The backend Azure Function and Azure Cosmos DB were cut down for two reasons:

- Theoretical Reason (Possible Unexpected Charges)
- Practical Reason (Backend Cold Start)

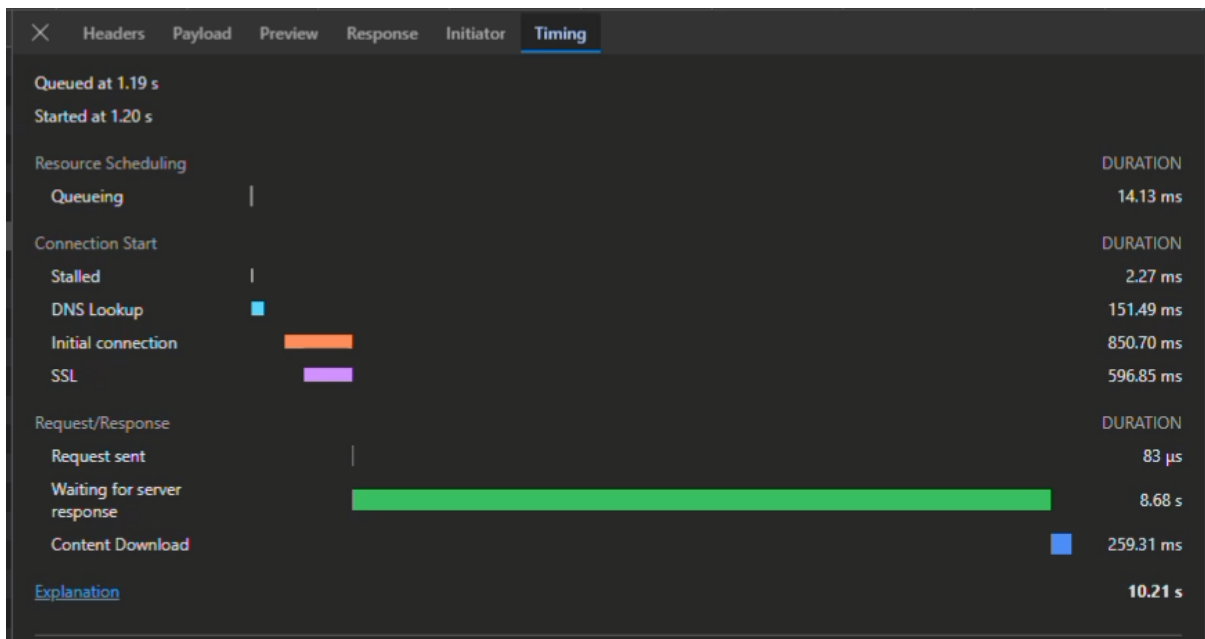
The flow is now:

- DNS Resolution: When a user navigates to the domain, Cloudflare resolves the request and directs traffic to the Azure Static Web App (SWA) origin.
- Client-Side Rendering: React processes the JSON data bundled together in its build to dynamically update the UI components for the user.



Theoretical Reason - There could be a possibility that a malicious script could loop and infinitely hit the website with many incognito tabs or cache-disabled browsers. This would cause a huge number of Azure Function HTTP Triggers and Azure Cosmos DB Request Units (RU) that would incur unexpected charge spikes. Although Azure Static Web App has built-in DDOS protection, it never hurts to be safe.

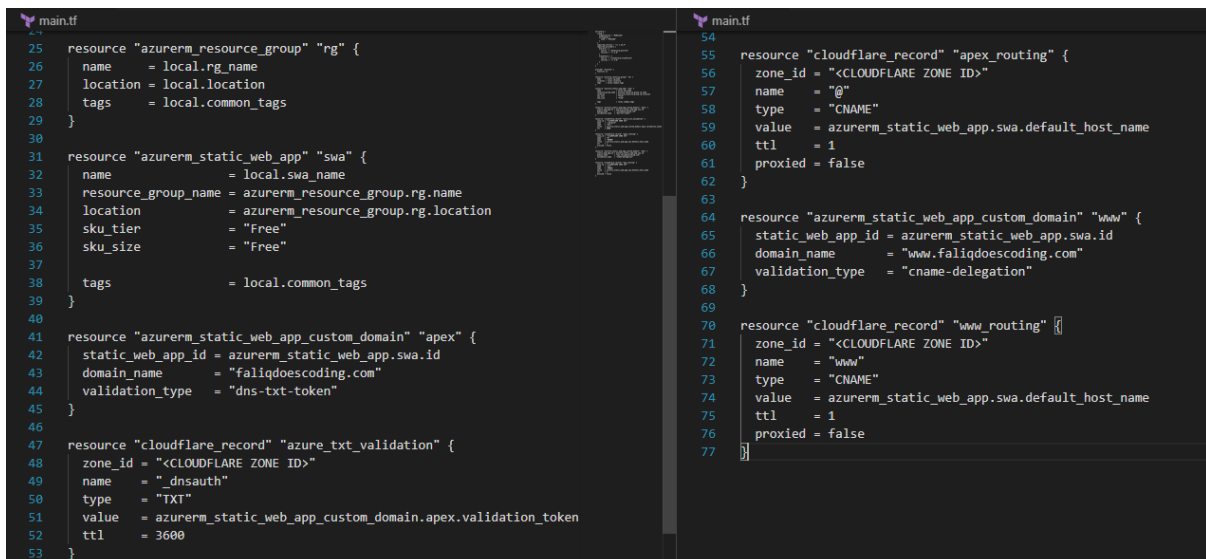
Practical Reason - When Azure Function has been idle for a while, Azure reclaims the underlying container resources. The next time a HTTP Trigger occurs, Azure has to allocate the container, start the function host, load the code and then initialise it again and this will take time. This is what is called a cold start and it disrupts the user experience. As seen in the image below, it took about 8 seconds to get a response from the server. In that time, the user could only see a React page with no images or text as those are all data stored in Azure Cosmos DB fetched by the Azure Function (that was idle).



Terraform

Compared to Phase 2, the Terraform configuration for this Phase is less complicated and fewer resources. Terraform only needs to deploy:

- 1 Resource Group
- 1 Azure Static Web App (SWA)
- 1 TXT Custom Domain in the SWA
- 1 CNAME Custom Domain in the SWA
- 1 TXT Record in Cloudflare
- 1 CNAME Subdomain (www) Record in Cloudflare
- 1 CNAME Apex Record in Cloudflare



```
main.tf
25 resource "azurerm_resource_group" "rg" {
26   name     = local.rg_name
27   location = local.location
28   tags     = local.common_tags
29 }
30
31 resource "azurerm_static_web_app" "swa" {
32   name                = local.swa_name
33   resource_group_name = azurerm_resource_group.rg.name
34   location            = azurerm_resource_group.rg.location
35   sku_tier            = "Free"
36   sku_size            = "Free"
37   tags                = local.common_tags
38 }
39
40 resource "azurerm_static_web_app_custom_domain" "apex" {
41   static_web_app_id = azurerm_static_web_app.swa.id
42   domain_name       = "faliqdoescoding.com"
43   validation_type   = "dns-txt-token"
44 }
45
46 resource "cloudflare_record" "azure_txt_validation" {
47   zone_id = "<CLOUDFLARE ZONE ID>"
48   name     = "_dnsauth"
49   type     = "TXT"
50   value    = azurerm_static_web_app_custom_domain.apex.validation_token
51   ttl      = 3600
52 }
53
main.tf
54 resource "cloudflare_record" "apex_routing" {
55   zone_id = "<CLOUDFLARE ZONE ID>"
56   name     = "@"
57   type     = "CNAME"
58   value    = azurerm_static_web_app.swa.default_host_name
59   ttl      = 1
60   proxied  = false
61 }
62
63 resource "azurerm_static_web_app_custom_domain" "www" {
64   static_web_app_id = azurerm_static_web_app.swa.id
65   domain_name       = "www.faliqdoescoding.com"
66   validation_type   = "cname-delegation"
67 }
68
69 resource "cloudflare_record" "www_routing" {
70   zone_id = "<CLOUDFLARE ZONE ID>"
71   name     = "www"
72   type     = "CNAME"
73   value    = azurerm_static_web_app.swa.default_host_name
74   ttl      = 1
75   proxied  = false
76 }
77 }
```

Afterwards the following commands were executed:

- terraform init (installs azure and cloudflare provider plugins)
- terraform validate (checks terraform configuration for syntax errors)
- terraform plan -out=plan.tfplan (creates an execution plan and informs what will happen when applying)
- terraform apply "plan.tfplan" (executes the execution plan created and deploys the configured resources)

rg-myresume-production

Resource group

Generate Bicep code to duplicate this resource group.

+2

✕

Search

◇

◀

+ Create

⚙️ Manage view ▾

🗑️ Delete resource group

🔄 Refresh

⋮

☰ Group by none ▾

Overview

Activity log

Access control (IAM)

Tags

Resource visualizer

Events

Settings

Cost Management

Monitoring

Automation

Help

Essentials

Subscription [\(move\)](#)

[Azure subscription 1](#)

Deployments

1 Succeeded

Subscription ID

Location

Central US

Tags [\(edit\)](#)

Environment : Production

ManagedBy : Terraform

Show more (1)

Resources

Recommendations

🔍 Filter for any field...

Type equals all ✕

Location equals all ✕

+ Add filter

☐

Name ↑

Type

Location

☐

 swa-myresume-production

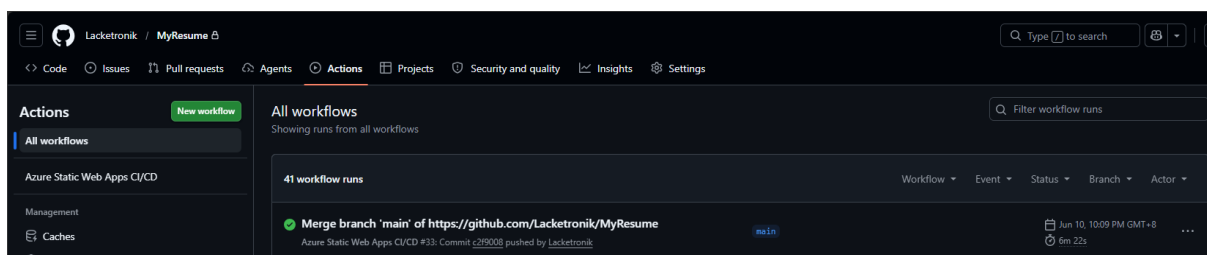
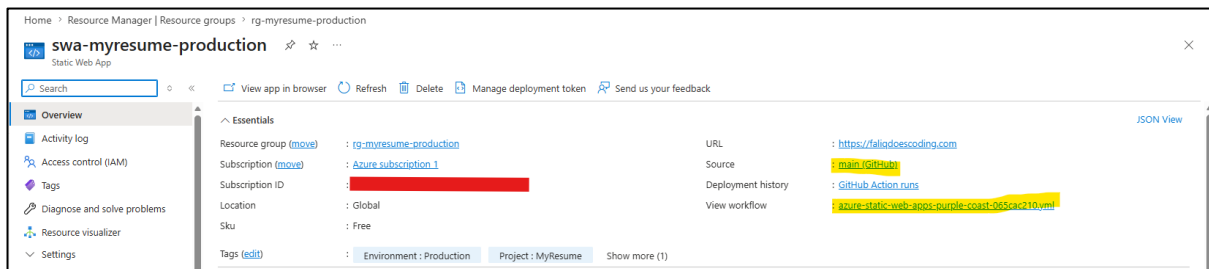
⋮ Static Web App

Central US

6

Azure Static Web App

After the azure Static Web App (SWA) is deployed, the main branch of the GitHub repo for MyResume was linked to it to establish a CI/CD pipeline. This means that any commits pushed to the main branch will trigger a GitHub workflow that will attempt to build the React application, deploy it to the SWA and serve it live.

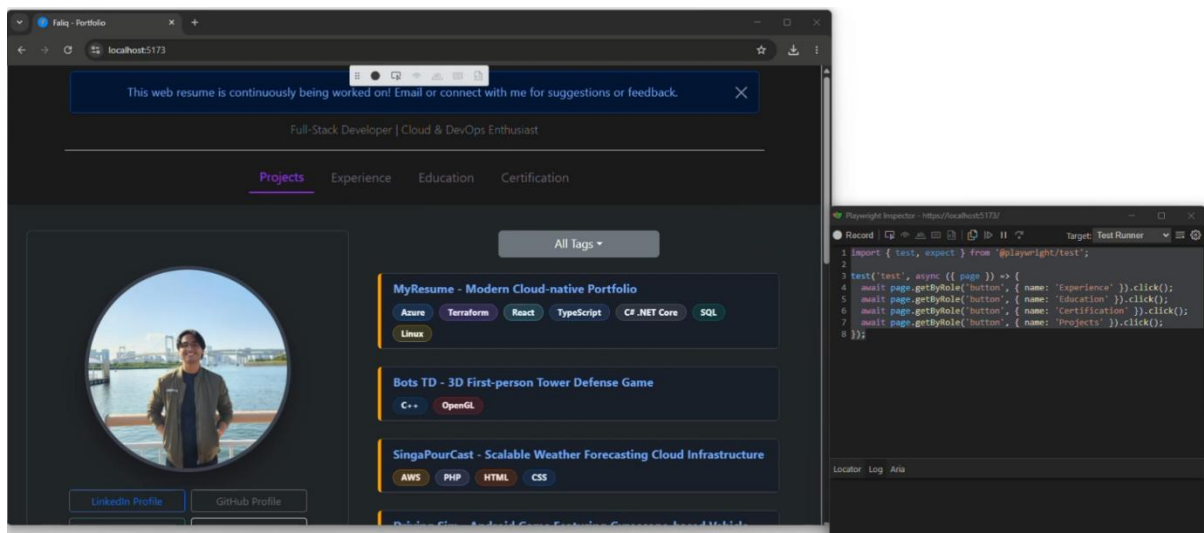


Playwright

To ensure the reliability of the UI, I have integrated Playwright into the development workflow. Playwright acts as an automated user, simulating real-world browser interactions to validate that the React components function as intended. The tests were conducted locally before it is then integrated together with the deployment workflow (more on that later)

After installing Playwright in the Web working directory, I first ran:

- npm run dev (to run the React SPA locally)
- npx playwright codegen <https://localhost:5173> (launches playwright UI in codegen mode)



Playwright codegen generates the testing code as I perform the actions on the target page. This code will 'simulate' the same action when running playwright test. I used the generated code as a baseline as I then added on logs, assertions and refinements to ensure they work across different browsers. The example above is a simple test code to navigate between the four tabs, Projects, Experience, Education and Certifications. The code below is the refined code of it to assert the expected outcome to ensure that everything is working as intended.

How the test works for example:

- Load the page
- Locate & Click on the 'Experience' tab
- Experience UI Component should have the word 'Responsibilities' and this is the expected outcome
- Logs the successful test & passes that test

```
TS navigationbar.spect...
tests > TS navigationbar.spect... test.describe('Navigation Tabs') callback > test('Test Projects Tab') callback
1 import { test, expect } from '@playwright/test';
2
3 test.describe('Navigation Tabs', () => {
4
5   test.beforeEach(async ({ page }) => {
6     await page.goto('https://localhost:5173/');
7   });
8
9   test('Test Experience Tab', async ({ page }) => {
10    await page.getByRole('button', { name: 'Experience' }).click();
11    await expect(page.getByText('Responsibilities').first()).toBeVisible();
12    console.log('Experience tab tested successfully.');
```

After the tests are configured, I ran:

`npx playwright test --ui`

To run the tests in a GUI and see the results firsthand.

The screenshot displays the Playwright Test GUI. On the left, a sidebar shows a list of test files and their status. The 'navigationbar.spect...' file is expanded, showing four tests: 'Test Experience Tab', 'Test Education Tab', 'Test Certification...', and 'Test Projects Tab'. All tests are marked as 'Passed'. The 'Test Projects Tab' is selected, and its details are shown in the main panel. The details include the test name, duration (10.9s), and a list of actions: 'Click' (1.2s) and 'Expect' (138ms). The 'Click' action is highlighted, showing the command: `getByRole('button', { name: 'Projects' })`. The 'Expect' action is also highlighted, showing the command: `expect(page.getByText('Azure').first()).toBeVisible()`. The main panel also shows a web browser view of the application under test, which is a resume page. The browser view shows a profile picture, a name, and a list of projects. The bottom panel shows the network log, which includes messages like '[vite] connecting...', '[vite] connected.', and 'Download the React DevTools for a better development experience: https://react.dev/link/react-devtools'. The log also shows a CORS error: 'Access to XMLHttpRequest at 'https://cloudflareinsights.com/cdn-cgi/rum' from origin 'https://localhost:5173' has been blocked by CORS policy: Response to preflight request doesn't pass access control check: The 'Access-Control-Allow-Origin' header has a value 'https://localhost' that is not equal to the supplied origin.'

After all testing is done and passed with satisfaction, Playwright is then ready to be integrated into the Deployment Workflow.

Deployment Workflow

The deployment automation after a commit is pushed to the main branch goes as follows:

- Install React Dependencies
- Start React Locally
- Start Playwright Tests
- Build Application
- Upload Build files & Deploy to Azure Static Web App



Playwright tests have to pass all tests before the project can be built. This is to ensure the reliability of the React components, verifying that they function as intended even when data is incorrect. A passed test report will be displayed as follows:

Q Search tests

All48

✓ Passed48

Failed0

Flaky0

Skipped0

> Metadata

10/06/2026, 10:11:26 pm Total time: 3.9m

▼ about-me-buttons.spec.ts

✓ Profile Links and Resume > Test LinkedIn Link

chromium

1.8s

about-me-buttons.spec.ts:9

✓ Profile Links and Resume > Test GitHub Link

chromium

1.3s

about-me-buttons.spec.ts:19

✓ Profile Links and Resume > Test Resume Download

chromium

819ms

about-me-buttons.spec.ts:29

✓ Profile Links and Resume > Test View Resume

chromium

796ms

about-me-buttons.spec.ts:38

✓ Profile Links and Resume > Test LinkedIn Link

chromium

3.7s

about-me-buttons.spec.ts:47

If the app is able to pass all the Playwright tests, the project will be built and then uploaded to the Azure Static Web App to be deployed live.

```
19  Zipping App Artifacts
20  Done Zipping App Artifacts
21  Uploading build artifacts.
22  Finished Upload. Polling on deployment.
23  Status: InProgress. Time: 0.1175296(s)
24  Status: Succeeded. Time: 15.2060581(s)
25  Deployment Complete :)
26  Visit your site at: https://purple-coast-065cac210.7.azurestaticapps.net
27  Thanks for using Azure Static Web Apps!
28  Exiting
```

Conclusion

MyResume has evolved from a traditional, manually managed infrastructure into a modern, robust and cost-efficient cloud-native solution. This four phase project successfully demonstrates the transition from a legacy architectural patterns to a modern serverless environment.

- Phase 1: Established competency in building a 3-tier application and understanding how the frontends backend APIs and RDBMS interact within a structured environment. Getting familiar with Azure Portal and is able to deploy resources manually.
- Phase 2: Transitioned from manual 'ClickOps' to Infrastructure as Code (IaC) using Terraform to ensure consistent, repeatable and version-controlled environment provisioning.
- Phase 3: Modernised the architecture by shifting to a serverless model with Azure Static Web Apps, Functions and Cosmos DB and gaining hands-on experience in cloud-native service integration and asynchronous data handling.
- Phase 4: Refined the infrastructure by implementing CI/CD pipelines and integrating automated quality assurance. Optimised for cost and performance by mitigating cold start latency and minimising unnecessary resource triggers.

Building this project has been an incredible learning experience. I have gone from manually configuring virtual machines to orchestrating a fully automated, serverless pipeline. I have gained a much deeper appreciation for how the pieces of cloud architecture fit together. This journey has shown me how much there is to explore in cloud computing and I am genuinely excited to apply these skills to my next project and see what else I can build going forward.